

Flagella-cascade investigation — changes, 2026-08-07

Continuation of `CHANGES_2026-08-06.md` (the FliT checkpoint, stoichiometry fix, nucleation/elongation, fliC 10x fix). This document covers everything since: the comprehensive diagnostic panel, the fliC stoichiometry feasibility check, the division-disabled completion diagnostic, and the multi-generation investigation that led to discovering a real, general limitation in the whole-cell model unrelated to flagella.

1. Comprehensive single-cell panel (12 panels, current cache)

File: `studies/flagella-02-transcription-regulation/run_comprehensive_panel.py` → `charts/08_comprehensive_panel_2400s.svg`

Why: no existing chart showed FlgM/FliA/FliT/FliD, the transcriptional overrides, free FliC, and the three ex-Gillespie reaction products (motor-switch, motor-complex, complete flagellum) all together, on the current, fully-fixed cache (`out/cache_full_flit_v4` , with the fliC 10x fix). Requested at Maya's explicit specification: "a more comprehensive panel in more detail... counts for the reactions we moved from the complexation.tsv... flgM/flia/flit/flid... the overrides, the flic counts."

Design note: FliA/FlgM and FliT/FliD/bound-complex are shown as combined overlays (each pair's own binding/sequestration dynamic is the biologically meaningful thing to see), not fully separated into individual panels — flagged as a design choice, not the only valid reading of the request.

Result (2400s, single cell): FlhD4C2 335→3 (FliT checkpoint working strongly), free FliT-dimer 974→169, FliT:FliD bound complex 0→926 (was flagged as an unexplained "always 0" observation in an earlier session turn — resolved: it wasn't stuck at zero, just hadn't had simulated time to build up), free FliC 51,982→10 (same depletion pattern as before), `CPLX0-7450` (motor switch) stayed exactly 0→0 the whole run while `FLAGELLAR-MOTOR-COMPLEX` went 0→2 — plausibly explained by same-tick sequential consumption (motor-switch-assembly runs immediately before motor-complex-assembly in the execution order) faster than the 30s sampling interval could catch a nonzero value; not independently confirmed.

2. FliC stoichiometry feasibility check (no code change — a calculation)

Context: Maya asked whether FliC's ~20,000-subunit-per-flagellum requirement should be reduced to make completions feasible within a reasonable simulated window.

Method: solved Renault et al. 2017's own growth law, $L(t) = \sqrt{b^2 + 2at} - b$ ($a=26,450$, $b=575$), for the time to reach 20,000 subunits assuming **unlimited** FliC (no synthesis bottleneck at all).

Result: 7,996.2s / 133.3 min (~2.22 hr) — the theoretical minimum, best case. A single ~42 min generation is shorter than this floor, so zero completions in any single-generation test is expected, not evidence the stoichiometry itself is unrealistic.

Recommendation (accepted): do not reduce the 20,000 coefficient — it's real, cited structural biology (PMC7696725), not a parameter choice. Either extend test windows or continue closing the fliC synthesis gap instead.

3. Division-disabled diagnostic: is completion an asymptotic stall, or just a timing problem?

Files: `studies/flagella-02-transcription-regulation/run_diagnostic_no_division.py` → `charts/09_diagnostic_no_division_18000s.svg`

Why: every single-cell test this session hits division at $t \approx 2,524\text{--}2,528\text{s}$ (~42 min, mass-threshold-adjacent D-period timer) and crashes (`KeyError: '0'` , agent replaced by daughters). This conflates two questions: (1) can the fliC-corrected calibration (~6/s, still ~7.6x short of ~46/s peak demand) ever complete a filament given enough time, or does growth asymptotically stall as more concurrent filaments compete for a still-limited pool; (2) does division itself prevent completion, independent of (1)? This diagnostic isolates (1).

Mechanism: monkeypatches `Division.next_update` to a no-op (`{}`) for the duration of one run only, fully reverted in a `finally` block. Verified via direct code research: `Division.update` just calls `self.next_update(1.0, state)` , so patching this one method suffices; no `config_overrides` path can reach `d_period` (hardcoded `True` in `Division.initialize` , not sourced from any override-able config).

Result: YES, completion is achievable. 9 flagella completed over 5 simulated hours (18,000s), first ones appearing between the 180 and 210-minute checkpoints — consistent with the 133-min analytical floor above. Once completions start, free FliC rebounds sharply (11→120,325) as elongation demand drops.

time	n_nascent	max_len	free FliC	completed_ever	dry_mass
30 min	4	9,198	4,383	0	595.9fg
120 min	8	14,381	1,292	0	1,887.6fg
210 min	12	19,842	32,651	3	5,760.8fg
300 min	15	19,883	120,325	9	18,130.6fg

Important caveat: division being disabled let `dry_mass` grow unbounded (ending ~18,131fg, ~26x a normal ~700fg division mass). Since ribosome/RNAP counts scale with cell size in this model, later completions (5-9, from ~240 min on) are likely inflated by this artificially oversized cell having far more synthesis capacity than any real dividing lineage would ever have at once. The **first** completions (~180-210 min, cell only ~8x oversized) are the cleaner signal.

Implication: under current calibration, no single real (dividing) cell can complete a flagellum within its own ~42 min lifetime. Completion, if it happens in the real model, can only come from inherited, in-progress construction (`nascent_flagellum` 's `filament_length`) accumulating across **multiple** real generations, via `divide_nascent_flagellum` 's whole-unit binomial inheritance (confirmed 2026-08-06 to survive division intact, continuing from its current length, not restarted).

4. Multi-generation investigation — discovered a general WCM limitation

This is the largest finding of the session. Testing whether inherited progress actually completes and converges to a realistic count (Sisti et al. 2017: 2.4-7.8 flagella/cell) over several real generations required tracking a lineage across many divisions. Three attempts:

4a. Manual splicing (first attempt) — a real, separate bug, since fixed by switching approach

File: `studies/flagella-02-transcription-regulation/run_lineage_multigen.py` (superseded version)

Copied `run_studies_multigen.py` 's existing generation-chaining pattern: call `divide_cell()` directly, rebuild a fresh `baseline()` composite, hand-splice `bulk` / `unique` / `environment` / `boundary` onto it.

Found a real, previously-undetected bug: `dry_mass` at division drifted sharply downward generation over generation (706.7 → 632.6 → 553.5 → ... → 262.0 fg by generation 7), eventually flooding the metabolism FBA solver with `GLP_NOFEAS` /numerical-instability warnings and driving complete-flagella count to 0. `run_studies_multigen.py` itself was never stress-tested past `--generations 2` (its only existing chart, `05_multigen_flgmg_flgia.svg`), so this compounding drift was never caught before.

Fix attempted: switch to the real `Division` step's own daughter-construction machinery (proven correct, no manual splicing), pruning to a single followed agent after each division — same linear compute cost, avoids the hand-rolled state transfer entirely.

4b. Real Division machinery + naive pruning — hit a framework bug

First attempt at the fix used `del comp.state["agents"][agent_id]` directly. Crashed on the very next `comp.run()`: `AttributeError: 'NoneType' object has no attribute 'get'` in `process_bigraph` 's `run_steps` . **Root cause:** `process_bigraph` 's `Composite` maintains internal step/process-path bookkeeping (`self.process_paths` / `self.step_paths`) that raw dict deletion leaves stale. **Fix:** use `comp.apply({"agents": {"_remove": [agent_id]}})` — the same `"_remove"` structural-update primitive `Division.next_update` itself uses (`division.py:439-447`), which internally calls `self.find_instance_paths(self.state)` to refresh the registry.

4c. Selection-bias check — ruled out

With the removal API fixed, the SAME monotonic drift reappeared (706.7→646.5→559.3→525.4 fg). Since “always keep the lexicographically first agent ID” could in principle correlate with a smaller share if `Division`’s daughter-ID assignment has any positional convention, switched to a genuine random coin-flip (its own RNG stream, decorrelated from ID string order) for which daughter to keep. **Drift persisted identically** (706.7→598.9→531.0→476.7→472.6 fg across 5 generations) — ~3% probability under pure chance, and combined with the previous run’s 4-decrease streak, this rules out selection-rule bias as the cause.

4d. Root cause investigation

Checked whether this was specific to single-lineage pruning by re-running the (already-validated) expanding-population approach (`run_flit_checkpoint_multigen.py`) for longer (4hr target instead of 2hr). **The identical pattern appeared** once it reached ~6 agents / generation 4 (divisions at 513.7, 565.0, 557.2, 521.5 fg + the same `GLP_NOFEAS` floods) — ruling out single-lineage-specific causes entirely.

Investigated and ruled out: shared/depleting environment state. Direct code inspection confirmed `ecoli_baseline` wires no `environment_mirror` /pooled-environment step (those only exist in `ecoli_colony.py` / `ecoli_time_varying_env.py`, unused here); each agent’s `environment` / `boundary` is a private, non-shared copy, explicitly carried at division (`division.py:148,306,388-390`); `media_update.py` is a no-op unless `media_id` itself changes; nothing decrements concentrations based on actual FBA uptake. So environment is a static, effectively-infinite value regardless of agent count or elapsed time — not the cause.

Most likely real cause (code-verified, not yet confirmed as fixable in-session): a compounding, per-cell feedback loop:

1. `division.py`’s default D-period timer (`self.d_period=True`) fires a *fixed* duration after chromosome replication completes — the dry-mass threshold is explicitly ignored on this path (`division.py:274-290`). There is no mechanism forcing an underweight cell to wait for adequate mass before dividing.
2. An underweight daughter (ordinary binomial variance in `divide_bulk` guarantees these occur routinely) divides at a proportionally smaller mass — it had the same fixed time to grow, no extra catch-up time.
3. That daughter’s metabolism FBA problem (calibrated around a ~700fg reference cell) becomes more likely to go infeasible.
4. `allocator.py` documents (by design) that an infeasible FBA tick can drive a bulk pool negative, and explicitly does **not** heal it afterward (“the negative pool is the upstream’s to heal”) — the corruption persists and is inherited unmodified by the next generation via ordinary `divide_bulk`.
5. That corrupted starting point makes the next generation grow more slowly, dividing even smaller — repeat, compounding.

This explains why it only appears after 3-4 generations (needs time to accumulate) and why it's identical whether tracking one lineage or the whole population (it's a per-agent property, not population-size-dependent).

Decision (Maya, 2026-08-07): document as a known limitation for now, do not attempt a fix in-session. This is a real, separate engineering effort (either adding mass-gated correction to the division timer, or making the allocator heal negative pools from infeasible solves), not a quick patch.

Practical implication for all future work in this repo, not just flagella: any multi-generation result should be treated as trustworthy only within roughly the first 3 generations (~2hr from a typical starting condition). Every result generated earlier in this session's work (the 2hr/4-agent population run, all single-cell tests, the division-disabled diagnostic) stayed within or used a controlled workaround for this window and remains valid. Longer, untested runs should be treated with suspicion until this is actually fixed.

Where this leaves the original question

The core question this investigation has been chasing — does inherited, in-progress flagellar construction complete and converge to a realistic count (2.4-7.8/cell) over several real generations — is **still open**. What's now known: - Completion is achievable given enough uninterrupted time (~180+ min per filament, confirmed directly). - A single generation (~42 min) can never be enough on its own — multi-generation inheritance is structurally required. - Testing that multi-generation hypothesis directly is currently blocked by the division/mass-homeostasis limitation above, unrelated to flagella, once pushed past ~3-4 generations.

Logged as open decisions in both `investigation.yaml` (`division-mass-homeostasis-gap` , the general limitation) and `studies/flagella-02-transcription-regulation/study.yaml` (`multigen-flagella-completion-still-untested-past-3-generations` , the flagella-specific consequence).